Course Name:

Introduction to Natural Language Processing

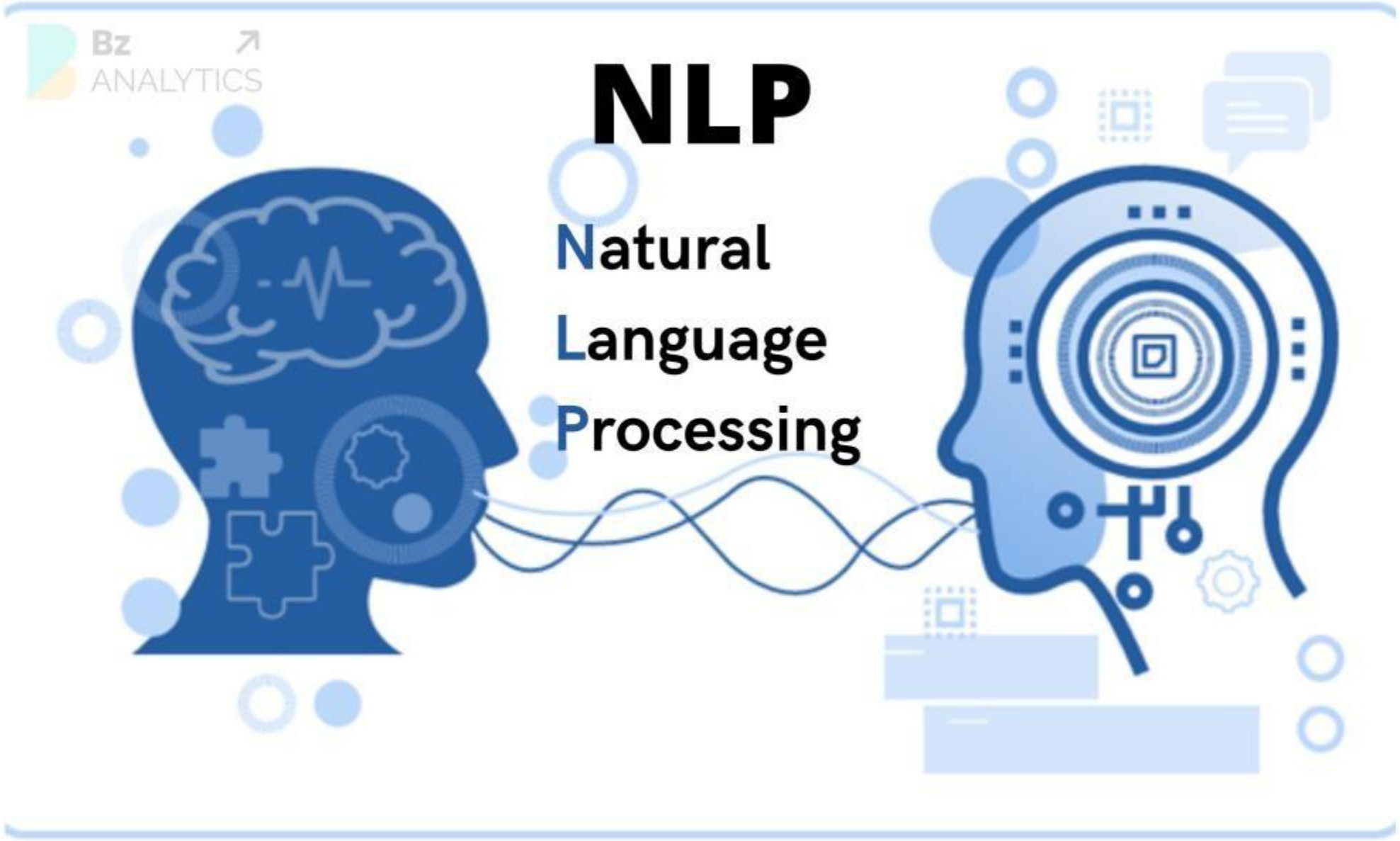
Course Objectives:

The objective of this course is to:

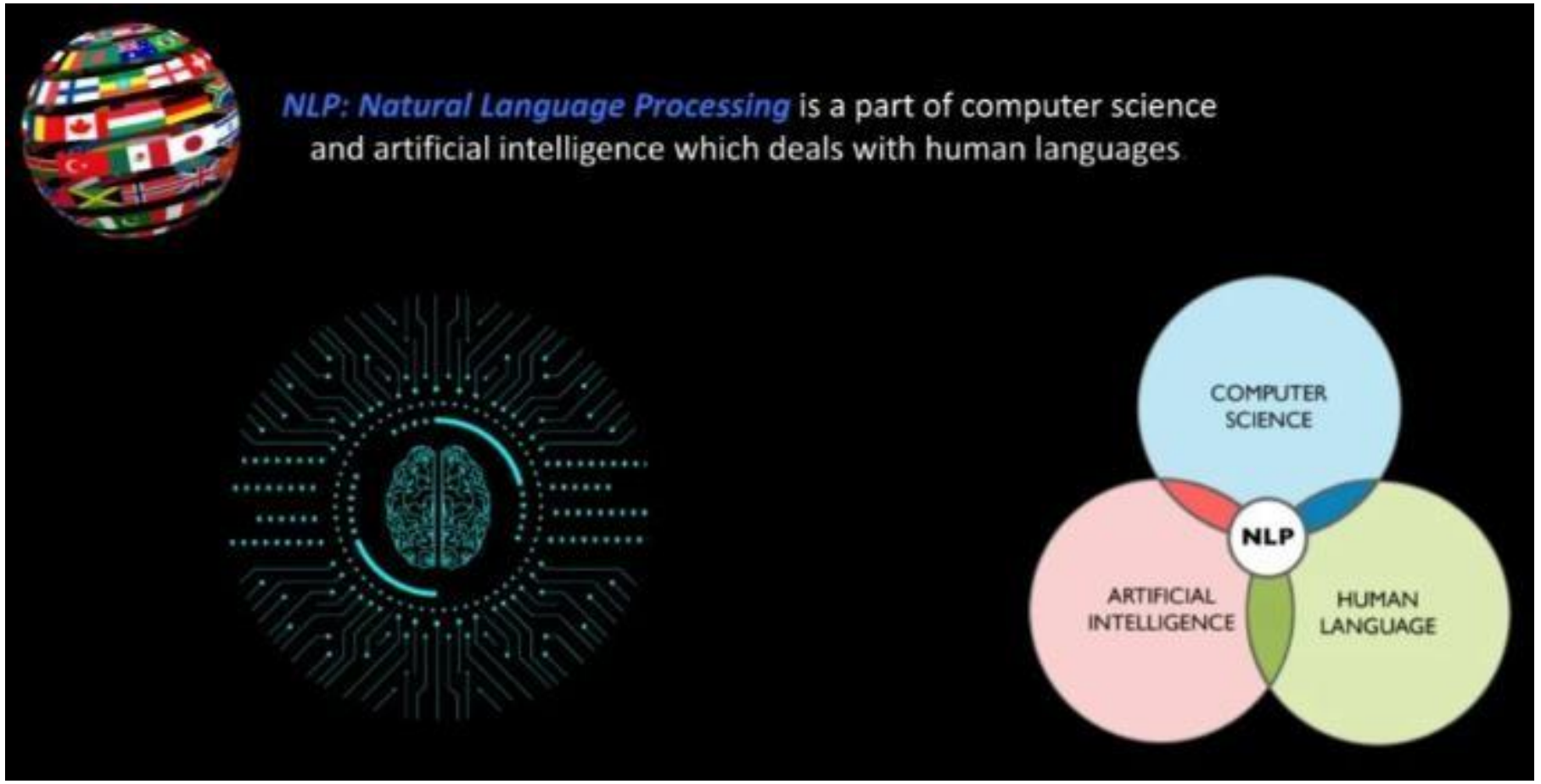
- To gain the basic knowledge on Natural Language Processing.
- Apply text cleaning, Morphological and Lexical analysis on text data.
- To enrich the algorithmic knowledge on the application of various syntactic and semantic parsing in NLP process.
- To grab the strong knowledge on the natural language generation in NLP process

Module -1 Introduction to Natural Language Processing

- Introduction to NLP and Text Processing
- Tokenization: Word and Sentence Tokenization
- Text Filtration and Script Validation
- Stop Word Removal Techniques
- Stemming and Lemmatization
- Text preprocessing pipeline (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)


Bz
ANALYTICS
NLP
Natural
Language
Processing

Natural Language Processing (NLP)



Text Mining and NLP

Text Mining / Text Analytics is the process of deriving meaningful information from natural language text

Natural Language Processing (NLP)

- **Natural Language Processing (NLP)** is a branch of Artificial Intelligence (AI) that enables computers to understand,

  interpret, and generate human language.

- Natural Language Processing (NLP) is a field of study that focuses on processing information contained in natural

  language text. It enables computers to understand, interpret, and generate human language.

- NLP is also known as:

✔ **Computational Linguistics (CL)**

✔ **Human Language Technology (HLT)**

✔ **Natural Language Engineering (NLE)**

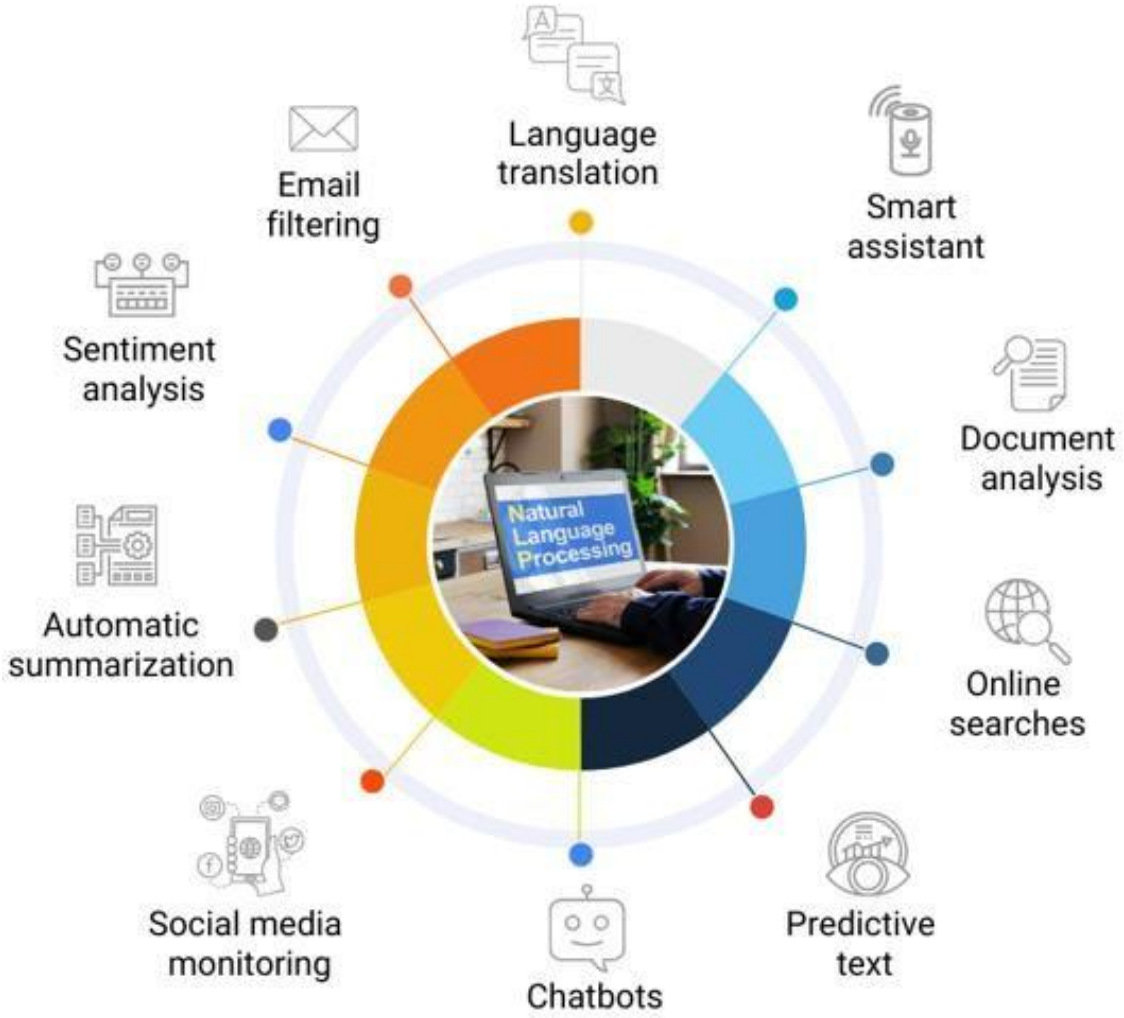
❑ Natural Language Processing (NLP) enables machines to:

- Analyze, understand, and generate human languages similarly to how humans do.
- Apply computational techniques to the field of language.
- Explain linguistic theories and use them to develop systems that benefit society.
- Evolve from its origins as a branch of Artificial Intelligence.
- Incorporate knowledge from Linguistics, Psycholinguistics, Cognitive Science, and Statistics.
- Make computers learn human language rather than requiring humans to learn machine language.

Importance of NLP in AI and Machine Learning

- Bridges the gap between human communication and machine understanding.
- Enables automation of text and speech-related tasks.
- Powers AI-driven applications like chatbots, search engines, and sentiment analysis.

Applications of Natural Language Processing



❑ Examples of NLP Applications

- ✔ **Chatbots & Virtual Assistants:** Siri, Alexa, Google Assistant.
- ✔ **Machine Translation:** Google Translate, DeepL.
- ✔ **Speech Recognition:** Voice commands in smart devices.
- ✔ **Text Analysis:** Sentiment analysis for customer reviews.
- ✔ **Summarization:** AI-generated article and report summaries.

Applications of NLP

1. Chatbots & Virtual Assistants

- AI-powered chatbots like **Siri, Alexa, Google Assistant** help users with queries, reminders, and automation.

- Customer service chatbots handle inquiries efficiently (e.g., banks, e-commerce).

2. Sentiment Analysis

- NLP is used to analyze emotions in text (positive, negative, neutral).

- Examples: **Analyzing social media posts, product reviews, and customer feedback.**

3. Machine Translation

- Automatically translates text between languages (e.g., **Google Translate, DeepL**).

- Helps in breaking language barriers for global communication.

4. Speech Recognition

•Converts spoken language into text.

•Used in **voice assistants, transcription software, and voice commands in smart devices.**

5. Information Retrieval (Search Engines)

•NLP helps search engines like **Google** understand and rank relevant content.

•Enables **semantic search** (understanding user intent beyond keywords).

6. Text Summarization

•AI-generated summaries of long documents, news articles, and reports.

•Example: **Automatic news summaries (Google News, SummarizeBot).**

Text Preprocessing in NLP?

- **Text preprocessing** is the backbone of any successful Generative or Natural Language Processing (NLP) project.

- It's the phase where raw text data undergoes various transformations to make it suitable for analysis and modeling.
- **Text preprocessing** in **Natural Language Processing (NLP)** is the process of cleaning and transforming raw text into a structured format that is easier for machines to analyze. Since raw text often contains noise, inconsistencies, and unnecessary elements, preprocessing helps improve the performance of NLP models.

- It is a crucial step in **Natural Language Processing (NLP)** to improve the performance of machine learning models.

Why text pre-processing is essential:

1. Cleaning and Normalization

- Removes unwanted characters, punctuation, and special symbols.
- Converts text to lowercase for uniformity.
- Handles spelling corrections and stopword removal to reduce noise.

2. Tokenization

- Splits text into words or sentences, enabling efficient processing.
- Helps in identifying meaningful units for further analysis.

3. Lemmatization and Stemming

- Reduces words to their root forms (e.g., "running" → "run").
- Improves consistency in text data by reducing word variations.

4. Stopword Removal

- Eliminates frequently occurring words (e.g., "the", "is") that do not add meaningful information.
- Reduces the dimensionality of data, improving processing speed.

5. Feature Extraction

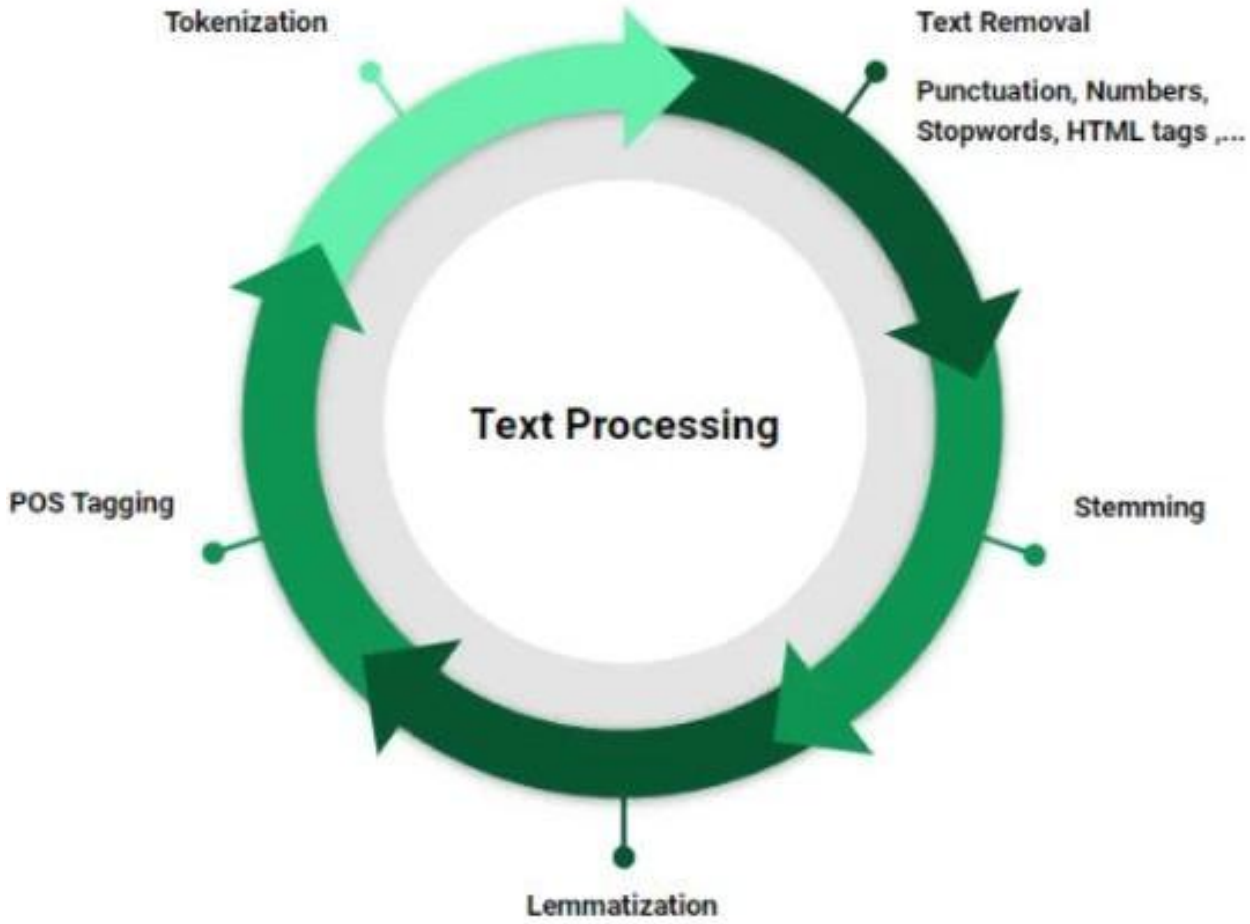
- Converts text into numerical representations like TF-IDF, Word Embeddings, or Bag of Words.
- Helps machine learning models understand text contextually.

6. Handling Ambiguity and Noise

- Detects and corrects spelling errors or typos.
- Resolves issues related to synonyms, polysemy (multiple meanings of a word), and homonyms.

7. Improves Model Accuracy

- Preprocessing enhances text quality, leading to better accuracy in NLP tasks such as sentiment analysis, machine translation, and text summarization.
- Without proper text processing, NLP models would struggle with inconsistencies, noise, and irrelevant information, reducing their effectiveness.


Tokenization
Text Removal
Punctuation, Numbers,
Stopwords, HTML tags ,...
Text Processing
POS Tagging
Stemming
Lemmatization

BAG OF WORDS

Bag of Words (BoW) is a basic text representation technique in NLP.
It converts text into numerical vectors using:

- Words present in text
- Frequency of words (count)
- It does NOT consider grammar or word order.

"Data Science is Science"
BoW counts:

•Data → 1
•Science → 2
•is → 1

Steps to Build BoW

Step 1: Collect documents (Corpus)

Step 2: Preprocessing

- Lowercasing
- Tokenization
- Remove punctuation
- Stopwords (optional)
- Stemming/Lemmatization (optional)

Step 3: Build Vocabulary (unique words)

Step 4: Convert each document into count vectors

Example Documents & Vocabulary

D1: “I love data science”
D2: “I love machine learning”
D3: “data science is powerful”
Vocabulary: [I, love, data, science, machine, learning, is, powerful]
BoW Vectors (Count Representation)
D1 → [1, 1, 1, 1, 0, 0, 0, 0]
D2 → [1, 1, 0, 0, 1, 1, 0, 0]
D3 → [0, 0, 1, 1, 0, 0, 1, 1]
Example 1: Count-Based BoW
Sentence: “AI is AI”
Vocabulary: [AI, is]
Vector: [2, 1]
Example 2: Binary BoW (0/1)
Sentence: “deep learning is fun”
Vector: [1, 1, 1, 1]

Example Documents & Vocabulary

Order Ignored Example
Sentence A: “I like ML”
Sentence B: “ML like I”
Vocabulary: [I, like, ML]
Both vectors → [1, 1, 1]

BoW in Sentiment Analysis (Example)
Sentence 1: “movie is good” → [1, 1, 1, 0] (Positive)
Sentence 2: “movie is bad” → [1, 1, 0, 1] (Negative)
Vocabulary: [movie, is, good, bad]

Limitations

Ignores word order (e.g., “not good” vs “good”)
No meaning/context (e.g., bank = river bank or money bank)
High dimensionality for large vocabularies

TOKENIZATION

- Unstructured text data, such as articles, social media posts, or emails, lacks a predefined structure that machines can readily interpret.

•Tokenization bridges this gap by breaking down the text into smaller units called tokens.

- These tokens can be words, characters, or even subwords, depending on the chosen tokenization strategy.

By transforming unstructured text into a structured format, tokenization lays the foundation for further analysis and processing.

What is Tokenization?

•**Tokenization** is the process of splitting text into smaller units called **tokens** (words, phrases, or sentences).

•It is the **first step** in many NLP applications, helping computers understand text structure.

WHY WE NEED TOKENIZATION???

•One of the primary reasons for tokenization is to convert textual data into a numerical representation that can be processed by
machine learning algorithms. With this numeric representation we can train the model to perform various tasks, such as

classification, sentiment analysis, or language generation.

- Tokens not only serve as numeric representations of text but can also be used as **features** in machine learning pipelines.

These features capture important linguistic information and can trigger more complex decisions or behaviors.

•For example, in text classification, the presence or absence of specific tokens can influence the prediction of a particular class.

Tokenization, therefore, plays a pivotal role in extracting meaningful features and enabling effective machine learning models.

Types of Tokenization

1. Word Tokenization

2. Sentence Tokenization

❑ **Word tokenization** splits a sentence or paragraph into individual words (tokens).

Examples:

- **Input:**

"Natural Language Processing is amazing!"

- **Output:**

["Natural", "Language", "Processing", "is", "amazing", "!"]

- **Input:**

"I love Natural Language Processing!"

- **Output:**

["I", "love", "Natural", "Language",
"Processing", "!"]

- **Sentence tokenization** **(also called sentence segmentation) breaks a paragraph or long text into individual sentences.**

Example:

- **Input:**

"Hello! How are you? I'm fine."

- **Output:**

["Hello!", "How are you?", "I'm fine."]

- **Input:**

"Natural Language Processing is fascinating. It helps computers understand text."

- **Output:**

["Natural Language Processing is fascinating.", "It helps computers understand text."]

- **Input:**

"NLP is amazing... but complex. (It's evolving fast!)"

- **Output:**

["NLP is amazing... but complex.", "(It's evolving fast!)"]

- Note: The parentheses and ellipses are handled correctly.

NLTK tool kit

NLTK is one of the oldest and most popular **Python libraries for NLP learning and research**.
It provides many built-in datasets and functions for text processing.

Main Features

✅ Tokenization (word/sentence splitting)

✅ Stemming & Lemmatization

✅ Stopword removal

✅ POS Tagging (Noun, Verb, etc.)

✅ Named Entity Recognition (basic)

✅ Text classification (basic ML support)

✅ WordNet (dictionary + synonyms)

TextBlob &spaCy

TextBlob is a **simple and beginner-friendly NLP library** built on top of **NLTK + Pattern**.
It provides easy one-line operations for common tasks.

Main Features
✅ Sentiment Analysis (Positive/Negative score)
✅ Part-of-Speech tagging
✅ Noun Phrase extraction
✅ Tokenization
✅ Translation (via Google translate API, needs internet)
spaCy is a **modern, fast, industry-level NLP library** designed for **real-world applications**.
It is much faster and more accurate than NLTK for many tasks.
Main Features

✅ Very fast tokenization
✅ POS tagging (high accuracy)
✅ Dependency parsing (sentence structure)
✅ Named Entity Recognition (NER)
✅ Word vectors
✅ Custom training pipelines
✅ Works well in production systems

Gensim

Gensim is a powerful NLP library mainly used for **topic modeling and word embeddings**.
It is not for tokenization or grammar tasks like spaCy/NLTK.
It focuses on **meaning-based text modeling**.

Main Features

✅ Topic Modeling (LDA)

✅ Word2Vec (word embeddings)

✅ Doc2Vec

✅ Similarity comparison between documents

✅ Works well on large text datasets

Comparison of NLP Toolkits

Aspect	NLTK	TextBlob	spaCy	Gensim
Primary Purpose	Academic learning and research	Simple and quick NLP tasks	Industrial and real-world NLP applications	Semantic modeling and topic analysis
Ease of Use	Moderate learning curve	Very easy and beginner-friendly	Moderate to advanced	Moderate (requires ML understanding)
Performance & Speed	Slower, not optimized for large data	Slower due to abstraction	Very fast and highly optimized	Efficient for large text corpora
Core Strength	Rich NLP tools and linguistic datasets	Simple sentiment analysis	Accurate NLP pipeline and NER	Topic modeling and word embeddings
Scalability	Limited scalability	Suitable for small datasets	Highly scalable	Designed for large-scale text
Typical Applications	Teaching, research, experimentation	Sentiment analysis, text polarity	Chatbots, IE systems, NLP products	Topic discovery, similarity analysis

Python Code for Tokenization

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize

# Download necessary resources
nltk.download('punkt')

# Sample text
text = "NLP is fascinating. It helps computers understand human language!"

# Sentence Tokenization
sentences = sent_tokenize(text)
print("Sentence Tokenization:", sentences)

# Word Tokenization
words = word_tokenize(text)
print("Word Tokenization:", words)
```

Challenges in Tokenization

- **Handling Punctuation:** "U.S.A. is a country." should be one entity, not "U", "S", "A" separately.
- **Dealing with Contractions:** "I'm" → "I", "am" (correct) vs. "I'm" (incorrect).
- **Multilingual Texts:** Some languages (e.g., Chinese, Japanese) don’t have spaces between words.

Text Filtration

Text filtration is the process of **removing unnecessary, irrelevant, or harmful content** from text data to improve the

quality and relevance for NLP applications.

Why is Text Filtration Important?

- Removes **stopwords, special characters, or profanity** to clean data.
- Helps in **spam filtering, content moderation, and privacy protection**.
- Enhances the efficiency of NLP models by reducing noise.

Methods of Text Filtration

- **Stopword Removal** – Removing common words like the, is, in, and that do not contribute much meaning.
- **Profanity & Offensive Content Filtering** – Detecting and replacing inappropriate words.
- **Duplicate & Redundant Data Removal** – Avoiding repeated words or sentences.
- **Special Character & HTML Tag Removal** – Cleaning unnecessary symbols like <html>, @, #, etc.

Example of Text Filtration

Input:

"I am very very happy!!! 😊 This is the best day <3 <html>."

After Filtration:

"I am happy. This is the best day."

❑ Stopwords

Stopwords are common words like *the, is, and, of* that don't add much meaning.

•Input:

"The quick brown fox jumps over the lazy dog."

•After Filtration:

"quick brown fox jumps over lazy dog."

❑ Filters out unnecessary symbols and emojis

Input:

"Hello!!! How are you??? 😊 #Excited"

After Filtration:

"Hello How are you Excited"

- **Replaces offensive words with asterisks or removes them.**

Input:

*"This is a **** stupid idea!"*

After Filtration:

*"This is a **** idea!"*

Simple Python Demo for Text Filtration

```python
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Download stopwords if not available
nltk.download('stopwords')
nltk.download('punkt')

# Sample text
text = "Hello! This is a demo for text filtration in NLP. Visit https://example.com for details

# Convert to lowercase
text = text.lower()

# Remove punctuation
text = re.sub(r'[^\w\s]', '', text)

# Remove URLs
text = re.sub(r'http\S+|www\S+', '', text)

# Tokenization
words = word_tokenize(text)

# Remove stopwords
filtered_words = [word for word in words if word not in stopwords.words('english')]

# Output
print("Filtered Text:", " ".join(filtered_words))
```

Expected Output:

```
Filtered Text: hello demo text filtration nlp visit details
```

Script Validation

Script validation ensures that the **text follows predefined rules** and is structured correctly before further NLP processing.

- **Why is Script Validation Important?**

- Ensures text is **well-formed and error-free**.
- Helps prevent **injection attacks, syntax errors, and invalid input** in applications like **chatbots, search engines, and form validation**.

- Validates **language-specific rules (e.g., Hindi vs. English scripts)**.

- **Script validation ensures that text input follows predefined rules and does not contain invalid, malicious, or unintended content.**

Key Aspects of Script Validation

1. Character Encoding Validation

Ensures correct character encoding (e.g., UTF-8, ASCII) to prevent errors in text

processing. Example:

Valid Input: *"Hello" (UTF-8 encoded)*

Invalid Input: *"Héllo" (if ASCII encoding is enforced)*

2. Language & Script Detection

Identifies the correct language script to prevent mixing of different languages in restricted

applications. Example:

Input: *"Bonjour! Comment ça va?" (French text in an English-only system)*

After Validation: *"Error: Non-English text detected!"*

3. Input Sanitization

Prevents malicious code injection (e.g., SQL injection, XSS attacks) by filtering harmful inputs.

Example:

Input: "SELECT * FROM users WHERE username='admin' --"

After Validation: "Invalid input detected!"

4. Grammar & Syntax Validation

Ensures that the text follows proper grammar and sentence

structure. Example:

Input: "He going to school"

Corrected Output: "He is going to school"

Python Demo for Script Validation

```python
import re
from langdetect import detect
import html

# Function to validate script
def validate_script(text):
    try:
        # Language detection (English expected)
        lang = detect(text)
        if lang != 'en':
            return "Error: Non-English text detected!"

        # SQL Injection Prevention
        if re.search(r"(SELECT|UPDATE|DELETE|INSERT|DROP|--)", text, re.IGNORECASE):
            return "Invalid input detected!"

        # HTML & JavaScript sanitization (XSS attack prevention)
        sanitized_text = html.escape(text)

        return f"Valid input: {sanitized_text}"

    except:
        return "Error in processing text!"

# Sample Inputs
print(validate_script("Hello, how are you?"))  # ✅ Valid input
print(validate_script("Bonjour! Comment ça va?"))  # ❌ Non-English detected
print(validate_script("SELECT * FROM users"))  # ❌ SQL Injection detected
```


Introduction to Natural Language Processing

1. Stop Word Removal in a Sentence

📌 Removes unnecessary words to retain only meaningful content.

Input Sentence:

📜 "The quick brown fox jumps over the lazy dog."

After Stop Word Removal:

✅ "quick brown fox jumps lazy dog."

2. Stop Word Removal in a Paragraph

📌 Cleans a longer text by filtering out stop words while keeping essential information.

Input Paragraph:

📜

"Natural language processing is a field of artificial intelligence that helps computers understand human language. It involves techniques such as tokenization, stop word removal, stemming, and lemmatization."

After Stop Word Removal:

✅

"Natural language processing field artificial intelligence helps computers understand human language. Involves techniques tokenization, stop word removal, stemming, lemmatization."

❑ Challenges & Considerations

- 🚧 Context Matters – Removing stop words may change the meaning (e.g., "To be or not to be" → "be not be").
- 🚧 Custom Stop Words Needed – Industry-specific texts (medical, legal, finance) require tailored stop word lists.
- 🚧 Multilingual Processing – Different languages need their own stop word lists.

Stemming in NLP

📌 Stemming is a rule-based process that removes suffixes to obtain the root form of a word.

📌 It does not consider meaning or context, which may result in incorrect words.

Example of Stemming

Word		After Stemming
Running	→	Run
Studies	→	Studi
Happily	→	Happili
Better	→	Better (Incorrect, should be "Good")

Popular Stemmer Algorithms:

Porter Stemmer

The Porter Stemmer is a simple algorithm that reduces words to their root form (stem) by removing common suffixes like "-ing", "-ed", and "-s". It's like a word-simplifier, helpful for text analysis and information retrieval.

Snowball Stemmer

The Snowball Stemmer, also known as the Porter2 Stemmer, is an effective stemming algorithm designed to process and reduce

words to their stems.

Lancaster Stemmer

The Lancaster Stemmer, also known as the Paice Stemmer, is a very aggressive algorithm used in natural language processing (NLP) to reduce words to their base or root form by removing suffixes, often resulting in shorter, sometimes non-existent words.

Lemmatization in NLP

- Lemmatization reduces a word to its dictionary form (lemma) by considering grammar and meaning. It requires a linguistic database (like WordNet) to find the correct base form.

Example of Lemmatization

Word		After Lemmatization
Running	→	Run
Studies	→	Study
Happily	→	Happy
Better	→	Good

Popular Lemmatization

- **Tools:** WordNet Lemmatizer
- (NLTK) spaCy Lemmatizer

What is Lemmatization?

	Stemming Operation	Lemmatization Operation
Preparing	Prepar	Prepare
Prepared	Prepar	Prepare
Prepares	Prepar	Prepare

Here's a simple Python code snippet demonstrating stemming and lemmatization using the NLTK library:

```
import nltk
from nltk.stem import PorterStemmer,
WordNetLemmatizer
from nltk.tokenize import
word_tokenize
# Download necessary
resources
nltk.download('punkt')
nltk.download('wordnet')
# Initialize stemmer and lemmatizer
stemmer = PorterStemmer()
lemmatizer =
WordNetLemmatizer()
# Example text
text = "The cats are running faster than the
dogs."

# Tokenize words
words = word_tokenize(text)

# Apply stemming
stemmed_words = [stemmer.stem(word) for word
in words]

# Apply lemmatization
lemmatized_words = [lemmatizer.lemmatize(word)
for word in words]

# Print results
print("Original Words:", words)
print("Stemmed Words:", stemmed_words)
print("Lemmatized Words:",
lemmatized_words)
```

What are Stemming and Lemmatization?

Both **stemming** and **lemmatization** are text normalization techniques in **Natural Language Processing (NLP)** used to **reduce**

words to their root forms. However, they work differently:

Feature	Stemming	Lemmatization
Definition	Reduces a word to its root form by removing suffixes.	Converts a word to its base form (lemma) using a dictionary.
Accuracy	Less accurate, may produce non- existent words.	More accurate, produces valid words.
Speed	Faster as it follows rule-based truncation.	Slower since it checks word meaning and grammar.
Use Cases	When speed is more important than accuracy.	When meaning and grammatical correctness are required.

Text Preprocessing Pipeline

A **Text Preprocessing Pipeline** is a sequence of steps used to clean and prepare raw text data for **Natural Language Processing (NLP)** tasks. This pipeline ensures that the text is in a structured and standardized format, making it suitable for analysis by machine learning models or linguistic algorithms.

Text Preprocessing Pipeline with the following key steps:

1. **Tokenization** – Splitting text into individual words or subwords.
2. **Filtration** – Removing unnecessary characters or symbols.
3. **Script Validation** – Ensuring valid characters and language script.
4. **Stop Word Removal** – Eliminating commonly used words that do not contribute to meaning (e.g., "is," "the," "and").
5. **Stemming** – Reducing words to their root form (e.g., "running" → "run").

This pipeline is commonly used in **Natural Language Processing (NLP)** for text analysis and machine learning applications.

REFERENCES

Text Books:

1. Foundations & Text Preprocessing

"Speech and Language Processing" – Daniel Jurafsky & James H. Martin

- Covers fundamental NLP concepts, text processing, POS tagging, parsing, and machine learning models.
- Best for understanding both theoretical and practical aspects of NLP.

"Natural Language Processing with Python" (NLTK Book) – Steven Bird, Ewan Klein, & Edward Loper

- Great for hands-on coding, especially for text preprocessing, POS tagging, chunking, and named entity recognition.
- Uses Python with NLTK, making it ideal for implementing your programs.

2. Morphological Analysis & Language Models

"Introduction to Natural Language Processing" – Jacob Eisenstein

- Covers morphology, syntax, and probabilistic language models (N-grams, HMMs).
- A good blend of theoretical concepts and programming examples.

3. Syntactic & Semantic Processing

"Handbook of Natural Language Processing" – Nitin Indurkhya & Fred J. Damerau

- Covers advanced syntactic analysis, POS tagging, chunking, and information extraction techniques.

"Statistical Natural Language Processing" – Christopher Manning & Hinrich Schütze

- Focuses on statistical approaches to NLP, including POS tagging, chunking, and language modeling.

4. Deep Learning & NLP Applications

"Deep Learning for Natural Language Processing" – Palash Goyal, Sumit Pandey, & Karan Jain

- Best for modern NLP applications like Named Entity Recognition, Transformers, and chatbot development. **"Natural Language Processing with Transformers"** – Lewis Tunstall, Leandro von Werra, & Thomas Wolf

- Focuses on deep learning and Transformer-based models (BERT, GPT, etc.).

THANK YOU